

שאלות הכנה למבחן בתכנות מערכות דפנסיבי

תוכן

2	יחידה 1: מבוא לתכנות דפנסיבי
4	יחידה 2: תכנות ++C
12	יחידה 3: התמודדות עם חולשות אבטחה
22	יחידה 4: תכנות דינאמי פייתון
28	יחידה 5: תקשורת
37	יחידה 6: מחשוב ענן ויישומי רשת
38	יחידה 7: SQL
44	סיכומים בנושאים שונים
44	סיכום פונקציות עם סיכון גבוה והתמודדות איתן
45	סיכום תהליכי מנגנוני הצפנה אי סימטרי
46	SSL/TLS
46	עמודים חשובים בחוברת:
46	אופרטור העברה והשמה
47	הבדל בין פונקציה רגילה ל־virtual

יחידה 1: מבוא לתכנות דפנסיבי

חלק א: שאלות רב-ברירה

שאלה 1

מהי ההגדרה הנכונה ביותר של "חולשת אבטחה" (Vulnerability)?

****מקור:** עמוד 8 במדריך הלמידה - הגדרת מושגי יסוד**

- א. כל באג בתוכנה שגורם לה להתנהג בצורה לא צפויה
- ב. טעות בכתיבת קוד המאפשרת למשתמש זדוני לבצע פעולות לא רצויות
- ג. תקיפת מערכת המתאפשרת בעקבות באג בקוד
- ד. הגנת המערכת בפני תקיפות
- תשובה: ב

שאלה 2

מה ההבדל העיקרי בין באג רגיל לחולשת אבטחה?

****מקור:** עמוד 8 במדריך הלמידה - הקשר בין באגים וחולשות אבטחה**

- א. חולשת אבטחה תמיד גורמת לקריסת התוכנה
- ב. באג תמיד ניתן לתיקון בעוד חולשת אבטחה לא
- ג. חולשת אבטחה מאפשרת לתוקף לבצע פעולות שלא אמור להיות מסוגל לבצע
- ד. אין הבדל מהותי בין השניים
- תשובה: ג

חלק ב: שאלות פתוחות

שאלה 3

הסבירו את המושג "צמידות חלשה" (weak coupling) בין מחלקות

****מקור:** עמודים 12-13 במדריך הלמידה - עקרונות עיצוב תוכנה**

צמידות חלשה היא עיקרון המתייחס למידת התלות בין מחלקות במערכת. במצב של צמידות חלשה, המחלקות מתקשרות זו עם זו דרך ממשקים מוגדרים היטב, תוך הסתרת פרטי המימוש הפנימיים שלהן.

"נקפיד להגדיר את כל השדות כפרטיים. זאת כדי להסתיר את פרטי המימוש מהמשתמשים במחלקה... הכוונה במוח 'הסתרה' כאן היא מיעת האפשרות של שימוש ישיר, ולא דווקא הסתרת המידע על אופן המימוש".

יחידה 2: תכנות ++C

שאלה 1: מנגנון הטבלה הוירטואלית

****שאלה:**** הסבר כיצד עובד מנגנון הטבלה הוירטואלית ב-C++. מה תפקידו של ה-vptr ואיפה הוא מאוחסן?

****מקור:**** מצגת עמודים 72-75, מיקוד עמ' 29-33

****תשובה:**** הטבלה הוירטואלית היא טבלה המכילה מצביעים לפונקציות עבור כל מחלקה. כל אובייקט מחזיק מצביע ("סודי") לטבלה זו הנקרא vptr או vpointer. הטבלה נוצרת על ידי הקומפילר כאשר מגדירים לפחות פונקציה אחת כ-virtual. ה-vptr מתעדכן בזמן יצירת האובייקט (בזמן ריצה) מיד אחרי הקריאה לבנאי של מחלקת האב.

שאלה 2: הבדל בין הכלה לירשה

****שאלה:**** מתי נשתמש בהכלה ומתי בירשה? הסבר את ההבדל בין שתי הגישות והדגם באמצעות דוגמא.

****מקור:**** מצגת עמודים 44-47, מיקוד עמ' 34-42

****תשובה:**** נשתמש בהכלה כאשר למחלקה "יש" (has-a) משהו, ובירשה כאשר המחלקה "היא" (is-a) סוג של משהו. לדוגמה: לבית "יש" חלונות (הכלה), בעוד שתפוח "הוא" סוג של פרי (ירשה).

שאלה 3: חריגות ב-C++

****שאלה:**** מהו ההבדל בין הטיפול בשגיאות בשפת C לבין הטיפול בשגיאות בשפת C++? מתי נעדיף להשתמש בחריגות?

****מקור:**** מצגת עמודים 83-89

****תשובה:**** בשפת C הטיפול בשגיאות נעשה בדרך כלל על ידי החזרת ערך מיוחד המייצג שגיאה, בעוד שב-C++ ניתן להשתמש במנגנון החריגות (exceptions). נשתמש בחריגות רק כאשר לא ניתן לטפל בשגיאה במקום בו היא התרחשה. החריגות מאפשרות לפעפע את הבעיה רמה אחת מעלה (לפונקציה הקוראת).

שאלה 4: כלל השלושה

****שאלה:**** הסבר מהו "כלל השלושה" ב-C++? מדוע הוא חשוב ומתי נצטרך ליישם אותו?

****מקור:**** מיקוד עמ' 43-44 במצגת 28 חוברת הקורס

****תשובה:**** כלל השלושה קובע שאם מממשים אחת מהפונקציות הבאות:

1. בנאי העתקה : מופעל כשיוצרים אובייקט חדש כהעתק עמוק של אובייקט קיים
 2. מפרק : אחראי לשחרור כל המשאבים שהוקצו
 3. אופרטור השמה
- a. מופעל כשמשייכים אובייקט קיים לאובייקט אחר
b. צריך לשחרר משאבים קיימים לפני ההעתקה

- יש לממש את שלושתן. הכלל חשוב במיוחד כאשר המחלקה מנהלת משאבי זיכרון דינמי.

הרחבה לכלל החמישה : נוסף עוד 2 מטודות :

4. באני העברה (Move Constructor) : מעביר משאבים מאובייקט זמני במקום להעתיק אותם
5. אופרטור השמה בהעברה (Move Assignment Operator) : מעביר משאבים מאובייקט קיים

שאלה 5: פולימורפיזם

****שאלה:**** מהו פולימורפיזם וכיצד הוא ממומש ב-C++?

****מקור:**** מצגת עמודים 63-65, מיקוד עמ' 34-42

****תשובה:**** פולימורפיזם הוא תכונה המאפשרת לטפל בערכים מטיפוסים שונים בעזרת ממשק אחיד. ב-C++, הוא ממומש באמצעות פונקציות וירטואליות וטבלה וירטואלית. המנגנון לא מופעל כברירת מחדל אלא יש להגדיר את הפונקציות כ-virtual.

שאלה 6: הרשאות גישה

****שאלה:**** מהן שלוש הרשאות הגישה ב-C++ ומה תפקידן?

****מקור:**** מיקוד עמ' 22-28

****תשובה:**** ישנן שלוש הרשאות גישה:

- public: נגיש לכולם

- private: נגיש רק בתוך המחלקה

- protected: נגיש במחלקה ובמחלקות יורשות

שאלה 7: חיתוך אובייקטים

****שאלה:**** מהי תופעת חיתוך האובייקטים (Object Slicing)? מתי היא מתרחשת וכיצד ניתן למנוע אותה?

****מקור:**** מצגת עמודים 78-79

****תשובה:**** חיתוך אובייקטים מתרחש כאשר מעתיקים אובייקט by-value ולא כמצביע בזמן ביצוע upcasting. כתוצאה מכך, המידע הייחודי של המחלקה היורשת נחתך ונשאר רק המידע של מחלקת הבסיס. ניתן למנוע זאת על ידי שימוש במצביעים או הפניות.

שאלה 8: ירושה מרובה

****שאלה:**** מהי בעיית היהלום בירושה מרובה? הדגם את הבעיה והסבר כיצד ניתן לפתור אותה.

****מקור:**** מצגת עמודים 60-62

****תשובה:**** בעיית היהלום מתרחשת כאשר מחלקה יורשת משתי מחלקות שיורשות מאותה מחלקת בסיס. הבעיה היא שעלולים להיווצר עותקים כפולים של מחלקת הבסיס. ניתן לפתור זאת באמצעות ירושה וירטואלית (virtual inheritance).

שאלה 9: תיאור תהליך הקריאה לפונקציה וורטואלית

תאר בפירוט את תהליך הקריאה לפונקציה וירטואלית בזמן ריצה. כיצד המערכת מזהה איזו פונקציה להפעיל?

מקור: מצגת עמודים 72-75

1. מבנה הטבלה הוירטואלית:

- כל מחלקה עם פונקציות וירטואליות מקבלת טבלה וירטואלית משלה
 - הטבלה מכילה מצביעים לכל הפונקציות הוירטואליות
 - כל כניסה בטבלה מצביעה לפונקציה הספציפית של המחלקה
2. תהליך הקריאה:
- כל אובייקט מכיל מצביע vptr בתחילתו
 - ה-vptr מצביע לטבלה הוירטואלית של המחלקה המתאימה
3. בזמן קריאה לפונקציה:
- גישה למצביע ה-vptr של האובייקט
 - גישה לטבלה הוירטואלית
 - גישה לפונקציה במיקום המתאים בטבלה
 - קריאה לפונקציה

דוגמא במספרים:

```
cppCopyShape* shape = new Circle;(5)
```

```
shape->getArea;()
```

הקומפיילר מתרגם את הקריאה ל:

- לקיחת ה-vptr (8 בתים ראשונים של האובייקט)
- גישה לטבלה הוירטואלית של Circle
- קריאה לפונקציה במיקום 0 (getArea)

שאלה 10

הסבר את היתרונות והחסרונות של שימוש בפונקציות וירטואליות והטבלה הוירטואלית. מדוע זה לא ברירת המחדל בשפת C++?

מקור: מצגת עמודים 67-68, מיקוד עמ' 34-42

תבנית תשובה מצופה:

1. פולימורפיזם בזמן ריצה:

- יכולת להחליף התנהגות בזמן ריצה
- טיפול אחיד באובייקטים מטיפוסים שונים
- הפרדה בין ממשק למימוש

2. גמישות בתכנון:

- יכולת להגדיר ממשקים מופשטים
- הרחבה קלה של המערכת
- תמיכה בעקרון ה-Open/Closed

3. תחזוקתיות:

- קל להוסיף מחלקות חדשות
- שינויים לוקליים בלבד
- קוד נקי יותר

חסרונות:

1. ביצועים:

- תקורה בזמן ריצה (חיפוש בטבלה)
- תקורת זיכרון (מצביע vptr לכל אובייקט)
- פחות יעיל מקריאות ישירות

2. מורכבות:

- קשה יותר להבין את זרימת התוכנית
- דיבוג מורכב יותר
- אפשרויות לבאגים נוספים

אכין מספר שאלות לדוגמה בנושא שגיאות קומפילציה וריצה ב-C++:

שאלה 11 למה הקוד לא מתקמפל

```
```cpp
class Shape{
 virtual double getArea() = 0; // private!
public:
 void printArea() {
 cout << "Area: " << getArea() << endl;
 }
};
```
```

פתרון: שגיאת קומפילציה משתי סיבות:

1. הפונקציה הוירטואלית `getArea` מוגדרת כ-private

2. במחלקה הנגזרת לא ניתן לדרוס פונקציה private

שאלה 12: מה הבעיה בקוד? האם זו שגיאת קומפילציה או ריצה

```
```cpp

// דוגמה 1: ללא וירטואליות - לא עובד כמצופה
class Base {
public:
 void process() { // פונקציה רגילה - לא וירטואלית
 cout << "Base processing" << endl;
 }
};

class Derived : public Base {
public:
 void process() { // override ולא דריסה (hiding) הסתרה
 cout << "Derived processing" << endl;
 }
};

Base* ptr = new Derived();
ptr->process ;()
```

```
//Base! יקרא לפונקציה של
'''
```

פתרון: אין שגיאת קומפילציה, אבל יש באג לוגי בזמן ריצה:

- הפונקציה `process` אינה וירטואלית

- תמיד תיקרא הפונקציה של מחלקת הבסיס

- נדרש להוסיף `virtual` כדי לקבל התנהגות פולימורפית והרצת הפונקציה של מחלקת הבן

---

**\*\*שאלה 13: איזה סוג שגיאה יש בקוד?\***

---

```
'''cpp
BadResource() {
 data = new int[100];
// הקצאת מערך
}
~BadResource() {

 delete data;
// delete שגיאה! צריך להיות []
}
};'''
```

פתרון: שגיאת ריצה - memory leak:

- הוקצה מערך עם `new` אבל משוחרר עם `delete` רגיל

- צריך להשתמש ב-`delete` לשחרור מערך

---

**\*\*שאלה 14: האם הקוד תקין? אם לא, מה הבעיה?\***

---

```
class BadAnimal {
```

```
public:
```

```

~BadAnimal() {
 cout << "BadAnimal destructor" << endl;
}

};

class BadDog : public BadAnimal {
private:
 string* name;
public:
 BadDog(const string& dogName) {
 name = new string(dogName);
 cout << "Created dog named " << *name << endl;
 }

 ~BadDog() {
 cout << "BadDog destructor - deleting " << *name << endl;
 delete name;
 }
};

int main() {
 cout << "=== virtual ===" << endl;
 {
 BadAnimal* bad_ptr = new BadDog("Rex");
 delete bad_ptr;
 // ייקרא destructor של BadAnimal רק!
 // לא משתחרר name - תוצאה: דליפת זיכרון
 }

 ...
}

```

פתרון: שגיאת תכנון שתגרום ל-memory leak:

- חסר destructor וירטואלי במחלקת הבסיס

- יגרום לדליפת זיכרון בשחרור דרך מצביע למחלקת הבסיס

---

**\*\*שאלה 15: מה הבעיה בקוד??\*\***

---

```

```cpp
void Update } ()

```

```
// שגיאת כתיב!
data = 42;
{
...

```

פתרון: שגיאת קומפילציה:

- הפונקציה נכתבה עם U גדולה במקום u קטנה
- זו פונקציה חדשה ולא דריסה של הפונקציה הוירטואלית
- המחלקה נשארת מופשטת כי לא מימשה את הפונקציה הוירטואלית הטהורה

****שאלה 6: מה הבעיה בקוד הבא?***

```
```cpp
class MyClass{
 MyClass(const MyClass& other){
 * this = other ;
 // שימוש באופרטור = לפני אתחול
 }
}

...

```

פתרון: שגיאה לוגית:

- שימוש באופרטור = לפני שהאובייקט אותחל כראוי
- עלול לגרום להתנהגות לא צפויה או קריסה

**יחידה 3: התמודדות עם חולשות אבטחה**

---

# שאלה 1: מהי גלישת חוצץ (Buffer Overflow) וכיצד היא יכולה להשפיע על אבטחת המערכת?

---

**\*\*מקור\*\*:** יחידה 3, עמודים 52-54 בחוברת הלימוד עמ' 13 - במצגת

**\*\*תשובה\*\*:**

- גלישת חוצץ היא חולשת אבטחה המתרחשת כאשר תכנית כותבת לאזור בזיכרון החוצץ יותר מידע מהגודל שהוקצה לו
- יכולה לגרום לדריסת מידע חשוב במחסנית כמו כתובות חזרה
- עלולה לאפשר הרצת קוד זדוני
- דוגמה נפוצה היא שימוש בפונקציה gets() שאינה בודקת גבולות

---

# שאלה 2: הסבר את מנגנון ה-ASLR ומדוע הוא חשוב להגנה מפני מתקפות?

---

**\*\*מקור\*\*:** יחידה 3, עמודים 39-40 במצגת

**\*\*תשובה\*\*:**

- ASLR (Address Space Layout Randomization) הוא טכניקה המאפשרת טעינת התהליך באזורים אקראיים בזיכרון
- מקשה על תוקפים לנחש כתובות בזיכרון
- מונע ניצול של חולשות כמו Return-Oriented Programming
- מספק שכבת הגנה נוספת גם אם קיימות חולשות אחרות בקוד

---

# שאלה 3: מהי התקפת Return-Oriented Programming (ROP) וכיצד היא עובדת?

---

**\*\*מקור\*\***: יחידה 3, עמודים 61-62 במצגת

**\*\*תשובה\*\***:

- שיטת תקיפה המאפשרת הרצת קוד זדוני ללא הכנסת קוד חיצוני למערכת
- התוקף משתמש בקטעי קוד קיימים ("גאדג'טים") ומשרשר אותם באופן זדוני
- מנצלת את מנגנון החזרה מפונקציות כדי לשלוט בזרימת התוכנית
- יעילה גם כאשר קיימים מנגנוני הגנה כמו DEP

---

# שאלה 4: הסבר על חשיבות בדיקת קלט מהמשתמש והדרכים להתמודד עם קלט לא צפוי

---

**\*\*מקור\*\***: יחידה 3, עמודים 32-34 במצגת וחוברת הלימוד

**\*\*תשובה\*\***:

- לעולם אין לסמוך על קלט מהמשתמש
- יש לבצע וולידציה מלאה של כל קלט
- שימוש בפונקציות בטוחות כמו fgets במקום gets
- הגבלת גודל הקלט בהתאם לגודל החוצץ שהוקצה
- שימוש במחלקות מובנות כמו string ב-C++ במקום מערכים של תווים

---

# שאלה 5: מהי קנרית המחסנית (Stack Canary) וכיצד היא מגנה על המערכת?

---

**\*\*מקור\*\*:** יחידה 3, עמודים 36-38 במצגת, ועמודים 57-59 בחוברת הלימוד

**\*\*תשובה\*\*:**

- קנרית המחסנית היא מנגנון הגנה מפני ניצול של גלישת חוצנהוא פועל על ידי הוספת ערך אקראי ("הקנרית") לפני כתובת החזרה במחסנית
- המערכת בודקת שהערך לא השתנה לפני החזרה מהפונקציה אם הערך השתנה, סימן שהייתה גלישת חוצץ והתוכנית מופסקת
- שם המנגנון מגיע מההיסטוריה של כורי פחם שהשתמשו בציפורים קנריות לגילוי גזים רעילים

---

# שאלה 6: מהי התקפת ערוץ צדדי (Side-channel Attack) וכיצד היא שונה מניצול חולשות קוד רגילות?

---

**\*\*מקור\*\*:** יחידה 3, עמודים 54-56 במצגת

**\*\*תשובה\*\*:**

- התקפת ערוץ צדדי היא התקפה המבוססת על מידע שמושג מהשימוש במערכת ולא דרך חולשות בקוד עצמו
- דוגמה קלאסית היא מדידת זמני הריצה של פונקציית השוואת סיסמאות
- התוקף לומד מידע על המערכת "מהצד" - למשל דרך צריכת חשמל, זמני ביצוע, או דליפות מידע אחרות
- ייחודה בכך שהיא יכולה לעקוף מנגנוני אבטחה חזקים בקוד על ידי ניצול המימוש הפיזי
- קשה יותר להגן מפני התקפות אלו כי הן מנצלות תכונות טבעיות של המערכת

---

# שאלה 7

---

בעת פיתוח מערכת בדיקת סיסמאות, איזו מההתנהגויות הבאות חושפת את המערכת להתקפת ערוץ צדדי?

**\*\*מקור\*\*:** יחידה 3, עמודים 55-56 במצגת

א. שמירת הסיסמאות בצורה מוצפנת

ב. יציאה מיידית כאשר מתגלה תו שגוי ראשון בסיסמה

ג. השוואת כל תווי הסיסמה ללא קשר להתאמה

ד. שימוש במלח (salt) בעת שמירת הסיסמאות

**\*\*התשובה הנכונה: ב\*\***

(כי זמן הריצה מגלה לתוקף מתי הניחוש נכון לתו הראשון)

---

### #שאלה 8

---

לפניך קטע קוד. איזו מההצהרות הבאות נכונה לגביו?

```
```cpp
void process_data(char* input){
    char buffer[256]
    strcpy(buffer, input);
{
```
```

**\*\*מקור\*\*:** יחידה 3, עמודים 51-54 בחוברת הלימוד

א. הקוד מוגן לחלוטין מפני גלישת חוצץ

ב. הקוד פגיע לגלישת חוצץ רק אם input ארוך מ-512 תווים

ג. הקוד פגיע לגלישת חוצץ בכל מקרה שבו input ארוך מ-256 תווים

ד. הקוד לא יכול לגרום לגלישת חוצץ כי הוא משתמש במצביעים

**\*\*התשובה הנכונה: ג\*\***

(כי strcpy לא בודק את גודל החוצץ היעד ויעתיק את כל התווים מהקלט)

---

### **\*\*שאלה 9 - ניתוח קוד\*\***

---

בדוק את הקוד הבא ומצא את חולשת האבטחה:

```
void vulnerable_function(char* input){
 char buffer[64]
 strcpy(buffer, input) ;
 // חולשה - אין בדיקת אורך
 {
```

מקור: שקופית 16-19 במצגת

חולשה: Buffer Overflow - אין בדיקה של אורך הקלט לפני ההעתקה ל-buffer. תוקף יכול להכניס קלט ארוך מ-64 בתים ולגרום לדריסת המחסנית.

---

**\*\*שאלה 10 - יישום\*\***

---

כתוב קוד המממש הגנה בסיסית מפני Buffer Overflow:

```
```c
void safe_process_input(const char* user_input){
    char buffer[64]
    size_t input_len = strlen(user_input);

    if (input_len >= sizeof(buffer)){
        printf("Error: Input too long\n");
        return;
    }

    strncpy(buffer, user_input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';
    {
```

מקור: שקופית 32 במצגת - "לעולם אל תסמכו על קלט מהמשתמש"

****שאלה 11 - ניתוח תרחיש****

זהה את סוג ההתקפה:

```
``` class Object {
 int* data;
public:
```

```
Object() { data = new int[10]; }
~Object() { delete[] data; }
};
```

```
Object* obj = new Object();
delete obj; // שחרור האובייקט
obj->data[0] = 42; // שימוש אחרי שחרור
```

מקור: שקופית 49-50 במצגת

תשובה: Use-After-Free - גישה לזיכרון אחרי ששוחרר. התקפה זו מסוכנת כי הזיכרון עלול לשמש כעת למטרה אחרת.

---

### שאלה 12 מנגנון DEP Data Execution Prevention

---

```
// קוד זדוני שמדפיס הודעה
char malicious_code[] = {
 0xB8, 0x21, 0x00, 0x00, 0x00, // mov eax, 33
 0xC3 // ret
};
```

```
int main() {
 void (*evil_func)() = (void(*)())malicious_code;
 evil_func(); // DEP ימנע את זה
 return 0;
}
```

שאלה: כיצד DEP מונע את ההתקפה הזו?  
תשובה: DEP מסמן אזורי זיכרון כ"לא להרצה" ומונע הרצת קוד מאזורי נתונים.

---

### שאלה 11 CET Control-flow Enforcement Technology

---

```
void* get_function_ptr() {
 return (void*)0x12345678 ;
} // ניסיון לקפוץ לכתובת שרירותית
```

```
{

int main() {
 void (*func)() = (void(*)())get_function_ptr;()
 func(); // CET לא חוקית קפיצה לא חוקית
```

שאלה: הסבר כיצד CET מגן מפני התקפות ROP?  
 תשובה CET: ע"י שהטכניקה שומרת על עותק עדכני של המחסנית בזיכרון לא נגיש  
 (בתמיכת החומרה). מוודא שקפיצות עקיפות מתבצעות רק לכתובות תקינות ומונע שימוש ב-  
 gadgets.

---

**\*\*שאלה 12 - ניהול זיכרון:\*\***

---

```
```cpp
class DataManager{
public:
    DataManager() {
        buffer = new char[100]
    }
    ~ DataManager() {
        delete buffer;
    }

    void setData(const char* data){
        strcpy(buffer, data);
    }
private:
    char* buffer;
};
```
```

שאלה: מצא לפחות 2 בעיות בניהול הזיכרון בקוד הנ"ל. כיצד ניתן לתקן אותן?

מקור: מופיע במבחן דוגמא 5, וגם בשקופיות על ניהול זיכרון נכון

תשובה:

1. בדסטרוקטור משתמשים ב-`delete` במקום `delete[]` עבור מערך
2. אין בדיקת אורך ב-`setData` שיכול לגרום ל-`buffer overflow`
3. חסר `copy constructor` ו-`assignment operator` (כלל השלושה/החמישה)

תיקון מוצע:

```

```cpp
class DataManager{
public:
    DataManager() : buffer(new char[100]){}
    ~ DataManager() {
        delete[] buffer;
    }
    DataManager(const DataManager& other){
        buffer = new char[100]
        strncpy(buffer, other.buffer, 99);
        buffer[99] = '\0';
    }
    void setData(const char* data){
        strncpy(buffer, data, 99);
        buffer[99] = '\0';
    }
private:
    char* buffer;
};
```

```

---

***\*\*שאלה 13- מבנה המחסנית:\*\****

---

```

```cpp
void process_user_input(char* input){
    char local_buffer[64]
    int access_flag = 0;
    strcpy(local_buffer, input);

    if(access_flag){
        printf("Access granted!\n");
    }
}
```

```

שאלה: הסבר כיצד תוקף יכול לנצל את הקוד הנ"ל כדי לקבל גישה. שרטט את מבנה המחסנית והסבר את מיקום המשתנים.

מקור: מוזכר במצגת בשקופיות 15-20 על מבנה המחסנית ו-buffer overflow

תשובה:

### מבנה המחסנית (מלמעלה למטה):

1. מצביע חזרה (Return Address)
    - כתובת בזיכרון אליה התכנית תחזור אחרי שהפונקציה תסיים
    - חיוני לזרימת התוכנית התקינה
  2. EBP שמור (Saved Base Pointer)
    - מצביע למסגרת המחסנית הקודמת
    - משמש לניווט בין מסגרות מחסנית שונות
  3. משתנים מקומיים (במקרה שלנו access\_flag)
    - משתנים שהוגדרו בתוך הפונקציה
    - נשמרים במחסנית
  4. באפרים (במקרה שלנו local\_buffer[64])
    - מערכים ומבני נתונים גדולים יותר
    - נמצאים בתחתית המסגרת
2. תוקף יכול לשלוח קלט ארוך מ-64 בתים שידרוס את access\_flag
3. דריסת access\_flag עם ערך שונה מ-0 תאפשר גישה

---

### **\*\*שאלה 14 - Integer Overflow\*\***

---

```

```cpp
size_t calculate_buffer_size(int num_elements, int element_size){
    return num_elements * element_size;
}

void* allocate_buffer(int num_elements, int element_size){
    size_t total_size = calculate_buffer_size(num_elements, element_size);
    return malloc(total_size);
}
```

```

שאלה: הסבר את הבעיה בקוד הנ"ל וכיצד תוקף יכול לנצל אותה. הצע תיקון.

מקור: מופיע במצגת בחלק על Integer Overflow ובמבחן דוגמא 3

תשובה:

1. הכפלה יכולה לגרום ל-integer overflow וזה מסוכן כי int משמש הרבה להקצאות וחסר הגנות
2. תוקף יכול לשלוח ערכים גדולים שיגרמו להקצאת זיכרון קטנה מדי
3. תיקון:

```

```cpp
void* allocate_buffer_safe(int num_elements, int element_size){
    if (num_elements <= 0 || element_size <= 0){
        return NULL;
    }
}
```

```

```
{

 // בדיקת גלישה
 if (num_elements > SIZE_MAX / element_size){
 return NULL;
 }

 size_t total_size = (size_t)num_elements * element_size;
 return malloc(total_size);
}
```

..

יחידה 4: תכנות דינאמי פייתון

---

# שאלה 1: דקורטורים ואבטחה

---

### נושא: מטא-תכנות ודקורטורים (מיקוד עמ' 45-46)

### השאלה (25 נקודות):

א. (10 נקודות) כתוב דקורטור `@log_access` שמבצע:

- רישום זמן הקריאה לפונקציה

- רישום שם המשתמש שקרא לפונקציה

- שמירת המידע לקובץ `log.txt`

ב. (8 נקודות) הדגם את השימוש בדקורטור על הפונקציה הבאה:

```
def get_user_data(username):
 return f"Data for user: {username}"
```

ג. (7 נקודות) הסבר כיצד הדקורטור מסייע באבטחת המערכת.

### פתרון:

```
```python  
from functools import wraps  
import time  
  
def log_access(func):  
    @ wraps(func)  
    def wrapper(username):  
        current_time = time.strftime("%Y-%m-%d %H:%M:%S")  
        with open('log.txt', 'a') as f:  
            f.write(f"{current_time}: Access to {func.__name__} by {username}\n")  
        return func(username)  
    return wrapper  
  
@log_access  
def get_user_data(username):  
    return f"Data for user: {username}"  
  
# הדגמת שימוש  
print(get_user_data("John"))
```

שאלה 2: SQL ואבטחת מידע

השאלה (25 נקודות):

א. (10 נקודות) כתוב פונקציה בפייתון המקבלת שם משתמש וסיסמה ומבצעת:

- חיבור לבסיס נתונים SQLite

- בדיקה האם המשתמש קיים

- החזרת סטטוס האימות

ב. (8 נקודות) הסבר כיצד להגן מפני SQL Injection בקוד שכתבת.

ג. (7 נקודות) כתוב דוגמה לקריאה בטוחה ולא בטוחה לפונקציה.

פתרון:

```
```python
import sqlite3

def verify_user(username, password):
 try:
 conn = sqlite3.connect('users.db')
 cursor = conn.cursor()

 # בטוח - שימוש בפרמטרים
 cursor.execute("SELECT * FROM users WHERE username = ? AND
password, '?' =
 (username, password)(
 result = cursor.fetchone()
 return result is not None

 except sqlite3.Error as e:
 print(f"Database error: {e}")
 return False
 finally:
 conn.close()
```

---

# שאלה 3: מחלקות ותכנות מונחה עצמים

---

### נושא: תכנות מונחה עצמים (מיקוד עמ' 78-82)

### השאלה (25 נקודות):

א. (10 נקודות) כתוב מחלקה בשם User המכילה:

- שם משתמש

- אימייל

- רמת הרשאה

- פונקציה לשינוי הרשאה

ב. (8 נקודות) כתוב מחלקה יורשת AdminUser שמוסיפה:

- יכולת לשנות הרשאות של משתמשים אחרים

- רשימת פעולות אחרונות שבוצעו

ג. (7 נקודות) הדגם שימוש במחלקות וכתוב דוגמה להעלאת הרשאה.

### פתרון:

```
```python
class User:
    def __init__(self, username, email, permission_level=1):
        self.username = username
        self.email = email
        self.permission_level = permission_level

    def change_permission(self, new_level):
        if new_level >= 1 and new_level <= 3:
            self.permission_level = new_level
            return True
        return False

class AdminUser(User):
    def __init__(self, username, email):
        super().__init__(username, email, permission_level=3)
        self.actions_log[] =
```
```

---

#### שאלה 4

---

כתבו בשפת פייתון מחלקה בשם MessageLogger הכוללת:

א. (8 נקודות) בנאי המקבל את שם הקובץ שבו יישמרו ההודעות.

ב. (8 נקודות) מתודה בשם add\_message המקבלת:

- חומרת ההודעה (error/warning/info)

- תוכן ההודעה

- זמן (אופציונלי, ברירת מחדל - הזמן הנוכחי)

ג. (9 נקודות) מתודה בשם get\_messages המקבלת פרמטר אופציונלי של חומרת הודעה ומחזירה את כל ההודעות מהסוג המבוקש. אם לא התקבל פרמטר, יש להחזיר את כל ההודעות.

שאלה 2 (25 נקודות)

מקור: עמ' 77-78 ביחידת הלימוד

הנתון הקוד הבא:

```
```python
```

```
class DataProcessor:
```

```
    def process(self, data):
```

```
        pass
```

```
class NumberProcessor(DataProcessor):
```

```
    def process(self, data):
```

```
        return sum(data)
```

```
class TextProcessor(DataProcessor):
```

```
    def process(self, data):
```

```
        return len(data.split())
```

```
def run_processor(processor_type, input_data):
```

```
    if processor_type == "number:"
```

```
        processor = NumberProcessor()
```

```
    elif processor_type == "text:"
```

```
        processor = TextProcessor()
```

```
    else:
```

```
        return None
```

```
    result = processor.process(input_data)
```

```
return result
```

```
...
```

- א. (10 נקודות) הסבירו את העיקרון של פולימורפיזם כפי שהוא בא לידי ביטוי בקוד הנ"ל.
- ב. (15 נקודות) הוסיפו מחלקה חדשה בשם ListProcessor שיורשת מ-DataProcessor ומבצעת את הפעולות הבאות על רשימה:
- מחזירה מילון שבו המפתח הוא טיפוס האיבר (str/int) והערך הוא כמות האיברים מאותו טיפוס
 - עדכנו את הפונקציה run_processor כך שתתמוך גם במעבד החדש

יחידה 5: תקשורת

1. מודל השכבות TCP/IP

שאלה א': "מה תפקידה של שכבת ה-Transport ומה ההבדל בין TCP ל-UDP?"

מקור: מצגת, שקופית 8

תשובה: שכבת ה-Transport אחראית על:

- העברת נתונים מקצה לקצה

- חלוקת מידע ליחידות קטנות במידת הצורך

- אבטחת הגעת המידע בסדר הנכון

ההבדל המרכזי: TCP מספק העברה אמינה עם אישורי קבלה, בעוד UDP מספק העברה מהירה ללא אימות.

שאלה ב': "מהו תפקיד פונקציית htons()?"

מקור: מבחן לדוגמא 2

תשובה: הפונקציה ממירה בין סדר בתים של המחשב המקומי (לרוב little-endian) לסדר הבתים המקובל ברשת (big-endian). חיונית במיוחד בעבודה עם מספרי פורט בתקשורת רשת.

2. תכנות Socket

שאלה א': "כתוב קוד בסיסי ליצירת שרת TCP המאזין בפורט 8080"

מקור: מבחן לדוגמא 1

```
python
import socket
```

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('127.0.0.1', 8080))
server_socket.listen(5)
conn, addr = server_socket.accept()
...
```

שאלה ב': "מהו הסדר הנכון של פעולות בצד השרת בתקשורת TCP?"

מקור: מצגת, שקופית 17

תשובה: הסדר הוא:

1. socket () - יצירת השקע
2. bind () - קישור לכתובת ופורט
3. listen () - האזנה לחיבורים
4. accept () - קבלת חיבור
5. send/recv - תקשורת
6. close () - סגירת החיבור

3. תקשורת מוצפנת:

שאלה א': "הסבר את תהליך החלפת המפתחות ב-SSL/TLS"

מקור: מצגת, שקופית 39

תשובה: התהליך כולל:

1. השרת מייצר זוג מפתחות (פרטי/ציבורי)
2. הלקוח מקבל את המפתח הציבורי
3. הלקוח מייצר מפתח סימטרי
4. הלקוח מצפין את המפתח הסימטרי עם המפתח הציבורי
5. השרת מפענח עם המפתח הפרטי
6. שניהם משתמשים במפתח הסימטרי להמשך התקשורת

שאלה ב': "מדוע משתמשים בשילוב של הצפנה סימטרית וא-סימטרית?"

מקור: מצגת, שקופית 38

תשובה: הצפנה א-סימטרית בטוחה יותר אך איטית. הצפנה סימטרית מהירה אך קשה להעביר את המפתח. השילוב מאפשר להעביר מפתח סימטרי בצורה מאובטחת (באמצעות הצפנה א-סימטרית) ואז להשתמש בו לתקשורת מהירה ויעילה.

4. תכנות מתקדם:

שאלה א': "כיצד ניתן לטפל במספר לקוחות במקביל בפייתון?"

מקור: מצגת, שקופית 37

תשובה: השימוש ב-selectors מאפשר:

- טיפול במספר חיבורים ללא צורך בתהליכים נפרדים
- ניהול יעיל של משאבי מערכת
- טיפול בפקודות קלט ללא חסימה
- מניעת מצב של הצפת חיבורים

שאלה ב': "מהם היתרונות של שימוש ב-boost לתקשורת ב-C++?"

מקור: מצגת, שקופית 30-32

תשובה: boost מספק:

- ממשק מופשט יותר לתכנות רשת
- טיפול אוטומטי בשגיאות
- תמיכה ב-cross-platform
- אפשרויות מתקדמות לתקשורת אסינכרונית

5. מהו SSL/TLS?

- SSL (Secure Sockets Layer) ו-TLS (Transport Layer Security) הם פרוטוקולים לאבטחת תקשורת

- TLS הוא הגרסה המתקדמת והמאובטחת יותר של SSL
- משמשים להצפנת התקשורת בין שני צדדים ברשת (למשל, בין דפדפן לשרת)

2. איך זה עובד?

התהליך כולל שני שלבים עיקריים:

א. **שלב "לחיצת היד" (Handshake):**

...

1. לקוח שולח "hello" לשרת
2. שרת מחזיר את התעודה הדיגיטלית שלו
3. לקוח מאמת את התעודה
4. הצדדים מסכימים על מפתח הצפנה משותף

ב. **שלב התקשורת המוצפנת:**

- כל המידע מוצפן באמצעות המפתח המשותף
- רק שני הצדדים יכולים לקרוא את המידע
- מבטיח סודיות ושלמות המידע

3. **היתרונות העיקריים:**

- הצפנת המידע
- אימות זהות השרת
- שלמות המידע (זיהוי שינויים)
- הגנה מפני התקפות man-in-the-middle

4. **דוגמה לקוד בסיסי ב-Python:**

עמ 125 חוברת לימוד

5. **שימושים נפוצים:**

- HTTPS (אתרי אינטרנט מאובטחים)
- דואר אלקטרוני מאובטח
- VPN
- תקשורת בנקאית
- העברת קבצים מאובטחת

1. תקשורת Socket בסיסית:

מקור: מבחן לדוגמא 2, שאלה 6

...

כתוב קוד בשפת ++C/C שיפתח לקוח תקשורת אל שרת שכתובת ה-IP שלו 119.4.7.5 בפורט 8080, וישלח לו הודעה. יש להדפיס את התשובה שמתקבלת.

עמ 111 בחוברת יש שרת ולקוח בשימוש BOOST

2. תקשורת מאובטחת (SSL/TLS):

מקור: מבחן לדוגמא 3, חלק מעשי

...

כתוב קוד בפייתון ליצירת שרת מאובטח המשתמש ב-SSL/TLS. על השרת להאזין בפורט 8443 ולהחזיר הודעת echo לכל הודעה שמתקבלת.

...

פתרון מוסבר:

עמ 125 בחוברת

3. עבודה עם מסד נתונים:

מקור: מבחן לדוגמא 4, שאלה 7

...

כתוב תכנית בפייתון שיוצרת טבלת Students במסד נתונים SQLite, מוסיפה נתונים ומאפשרת חיפוש. יש להתייחס לאבטחת SQL Injection.

...

פתרון מוסבר:

```
```python
import sqlite3
from sqlite3 import Error

def create_db_and_table():
 try:
 conn = sqlite3.connect('server.db')
 cursor = conn.cursor()

 # Prepared Statement יצירת טבלה עם
 cursor.execute("""
 CREATE TABLE IF NOT EXISTS Students(
 Name TEXT NOT NULL,
 Id TEXT PRIMARY KEY,
 Major TEXT NOT NULL
)
 """)

 # הוספת נתונים בצורה בטוחה
 students_data = [
 ('John Doe', '123456789', 'Computer Science'),
 ('Jane Smith', '987654321', 'Physics')
]
 cursor.executemany("""
 INSERT OR REPLACE INTO Students (Name, Id, Major)
 VALUES(?, ?, ?)
 """, students_data)

 conn.commit()
```

```

parametrized query
חיפוש בטוח עם
def search_student(name):
 cursor.execute("""
 SELECT * FROM Students
 WHERE Name? =
 """,
 (name,))
 return cursor.fetchone()

דוגמה לשימוש
result = search_student('John Doe')
print(f"Found student: {result}")

except Error as e:
 print(f"Database error: {e}")
finally:
 if conn:
 conn.close()

if __name__ == '__main__':
 create_db_and_table()
'''

```

---

#### 4. ניהול מספר לקוחות (Selectors):

---

מקור: מצגת, שקופית 37

'''  
 כתוב שרת בפייתון המסוגל לטפל במספר לקוחות במקביל באמצעות selectors.  
 '''

פתרון מוסבר:

```

'''python
import selectors
import socket
import types

```

```

def create_multi_client_server():
 sel = selectors.DefaultSelector()

 לקבלת נתונים callback # הגדרת פונקציית
 def accept_connection(sock, mask):
 conn, addr = sock.accept()
 conn.setblocking(False)
 data = types.SimpleNamespace(addr=addr, inb=b'', outb=b'')
 sel.register(conn, selectors.EVENT_READ | selectors.EVENT_WRITE,
 data=data)

 לטיפול בנתונים callback # הגדרת פונקציית
 def service_connection(key, mask):
 sock = key.fileobj
 data = key.data

 if mask & selectors.EVENT_READ:
 recv_data = sock.recv(1024)
 if recv_data:
 data.outb += recv_data
 else:
 sel.unregister(sock)
 sock.close()

 if mask & selectors.EVENT_WRITE:
 if data.outb:
 sent = sock.send(data.outb)
 data.outb = data.outb[sent:]

 # יצירת שרת
 server_socket = socket.socket()
 server_socket.bind(('127.0.0.1', 8080))
 server_socket.listen(100)
 server_socket.setblocking(False)

 # רישום השרת ל selector
 sel.register(server_socket, selectors.EVENT_READ, data=None)

 try:
 while True:

```

```
events = sel.select(timeout=None)
for key, mask in events:
 if key.data is None:
 accept_connection(key.fileobj, mask)
 else:
 service_connection(key, mask)
except KeyboardInterrupt:
 print("Shutting down")
finally:
 sel.close()

if __name__ == '__main__':
 create_multi_client_server()
...
```

## יחידה 6: מחשוב ענן ויישומי רשת

---

### 1. תכנות מקבילי:

---

שאלה (מבחן דוגמא 4, מותאם):

מדוע הקוד הבא לא בטוח במצב של ריצה מקבילית?

```
int counter = 0;
void increment() {
 counter++;
}
```

א. אין בעיה, הקוד תקין

ב. הפעולה ++counter אינה אטומית

ג. חסר הגדרת counter כ-volatile

ד. לא ניתן לשתף משתנים בין חוטים

תשובה: ב' - הפעולה ++counter אינה אטומית. היא מורכבת משלושה שלבים: קריאה, הגדלה, כתיבה. במצב של ריצה מקבילית, שני חוטים יכולים לקרוא את אותו ערך, להגדיל אותו, ולכתוב את אותו ערך בחזרה, מה שיגרום לאובדן עדכון.

---

### 2. תקשורת:

---

שאלה (מבחן דוגמא 3):

הפרוטוקולים הבאים פועלים ברמת ה-Transport:

א. Ethernet, Token Ring, FDDI

ב. TCP, UDP

ג. HTTP, SMTP, FTP

ד. IP, IPX, DECNET

תשובה: ב' - TCP ו-UDP הם פרוטוקולי שכבת התעבורה (Transport). הם אחראים על העברת המידע מקצה לקצה, כאשר TCP מספק העברה אמינה ו-UDP מספק העברה מהירה ללא אמינות מובטחת.

---

### 3. מחשוב ענן ואבטחה:

---

שאלה (מבוססת על מבחן דוגמא 5):

א תאר את מנגנון ה-Sandbox ואת מטרתו העיקרית. כיצד הוא מסייע באבטחת מערכות ענן?

תשובה :

Sandbox הוא מנגנון אבטחה המאפשר הרצת קוד בסביבה מבודדת תוך:

- הגבלת גישה למשאבי מערכת
- בידוד מהמערכת המארחת
- הגבלת הרשאות וגישה לרשת
- ניטור פעולות
- הוא חיוני במחשוב ענן כי הוא מאפשר הרצת קוד לא מוכר באופן בטוח

## יחידה 7: SQL

---

### 1. (25 נקודות) כתוב תכנית בשפת Python או ++C ופקודות SQL אשר:

---

א. (10 נק') ניגשת לקובץ בסיס נתונים SQLite בשם 'courses.db' ומוסיפה טבלה בשם Courses עם העמודות: CourseID, CourseName, Credits

ב. (10 נק') מקבלת מהמשתמש שם קורס ומדפיסה את מספר נקודות הזכות שלו

ג. (5 נק') הסבר איזו חולשת אבטחה עלולה להיות בקוד שכתבת וכיצד ניתן למנוע אותה  
מקור במצגת: עמודים 19-21 שמציגים את הפקודות הבסיסיות של SQL ועמוד 24 שמדבר על הזרקות SQL.

```
import sqlite3
```

```
א - יצירת הטבלה
```

```
def create_courses_table():
```

```
 try:
```

```
 conn = sqlite3.connect('courses.db')
```

```
 cursor = conn.cursor()
```

```

cursor.execute("""
CREATE TABLE IF NOT EXISTS Courses)
 CourseID INTEGER PRIMARY KEY,
 CourseName TEXT NOT NULL,
 Credits INTEGER
""")

```

```

 conn.commit()
except sqlite3.Error as e:
 print(f"שגיאה: {e}")
finally:
 conn.close()

```

```

ב - חיפוש נקודות זכות לפי שם קורס
def get_course_credits(course_name):
 try:
 conn = sqlite3.connect('courses.db')
 cursor = conn.cursor()

```

```

 # SQL Injection שימוש בשאילתה פרמטרית למניעת
 cursor.execute("SELECT Credits FROM Courses WHERE CourseName = ?",
(course_name,))
 result = cursor.fetchone()

 if result:
 return result[0]
 return None

except sqlite3.Error as e:
 print(f"שגיאה: {e}")
 return None
finally:
 conn.close()

```

# ג - החולשה היא SQL Injection והפתרון הוא שימוש בשאילתות פרמטריות

```

'''sql
query1 = "SELECT * FROM users WHERE userid = " + userInput;"" +
query2 = "SELECT * FROM users WHERE userid;"? =
'''

```

תשובה לשאלה 2:

- query1 פגיע ל-SQL Injection כי הוא משרשר מחרוזות באופן ישיר
- query2 משתמש בשאילתה פרמטרית (prepared statement) שמונעת SQL Injection
- ההבדל המהותי הוא שב-query2 הערך מועבר כפרמטר ולא כחלק מהשאילתה עצמה

---

### 3. (15 נקודות) ניתן הקוד הבא:

---

```

'''python
username = input("Enter username: ")
query = f"SELECT * FROM Users WHERE username = '{username}'"
'''

```

א. הסבר כיצד תוקף יכול לנצל את הקוד לרעה

ב. הצע תיקון שימנע את הבעיה

ג. האם השימוש ב-f-string בפייתון מגביר את הסיכון? הסבר

3. תשובה לשאלה 3: א. תוקף יכול להזין:

sql

' OR '1'='1

שיגרום לשאילתה להחזיר את כל המשתמשים

ב. תיקון:

python

```

cursor.execute("SELECT * FROM Users WHERE username = ?", (username,))

```

ג. השימוש ב-f-string מגביר את הסיכון כי הוא מבצע שרשרת מחרוזות בדומה ל-+

מקור במצגת: עמודים 24-25 העוסקים בהזרקות SQL ודרכי ההגנה

---

4. (20 נקודות) כתוב קוד בפייתון שקורא קובץ CSV המכיל נתוני סטודנטים (שם, ת"ז, ממוצע) ומכניס אותם לטבלת SQLite. עליך:

---

1. לוודא שאין כפילויות
2. לטפל בשגיאות אפשריות
3. להשתמש בשאילתות פרמטריות
4. להדפיס סיכום של מספר הרשומות שהוכנסו

```
import sqlite3
import csv

def import_students():
 conn = sqlite3.connect('students.db')
 cursor = conn.cursor()

 # יצירת טבלה
 cursor.execute("""
 CREATE TABLE IF NOT EXISTS Students(
 id TEXT PRIMARY KEY,
 name TEXT,
 average REAL
)""")

 inserted = 0
 with open('students.csv', 'r') as file:
 csv_reader = csv.DictReader(file)
 for row in csv_reader:
 try:
 cursor.execute("""
 INSERT OR IGNORE INTO Students (id, name, average)
 VALUES(?, ?, ?)
 """, (row['id'], row['name'], float(row['average'])))
 if cursor.rowcount > 0:
 inserted += 1
 except sqlite3.Error as e:
 print(f"שגיאה בהכנסת רשומה: {e}")

 conn.commit()
 print(f"הוכנסו {inserted} רשומות חדשות")
 conn.close()
```

---

## שאלה 5

---

1. איזו משאילתות ה-SQL הבאות מוגנת מפני SQL Injection?

- i. `"SELECT * FROM Users WHERE name = " + username`"` +
- ii. `"SELECT * FROM Users WHERE name = " + username``
- iii. `"SELECT * FROM Users WHERE name = ?", [username]``
- iv. ``"SELECT * FROM Users WHERE name = {username}"``

התשובה הנכונה: ג

הסבר: שימוש בשאילתה פרמטרית (?) עם מערך פרמטרים הוא הדרך הבטוחה למניעת SQL Injection.

מקור במצגת: עמוד 24

---

## 2. מה מטרת הפקודה `htons(PORT)` בקוד תקשורת?

---

- א. מוודאת שהשרת מכיר את מספר הפורט
- ב. ממירה בין Little-Endian ל-Big-Endian
- ג. מאפסת את 8 הביטים הראשונים במספר
- ד. ממירה חבילה בין IPv4 ל-IPv6

התשובה הנכונה: ב

הסבר: הפונקציה ממירה בין סדרי בתיים שונים להבטחת תאימות בתקשורת.

מקור במצגת: מופיע בדוגמאות הקוד בעמוד 15

---

## 3. מה ההבדל בין DML ל-DDL ב-SQL?

---

- א. DML לתחזוקת נתונים, DDL למחיקת נתונים
- ב. DML לטיפול בנתונים (SELECT, INSERT), DDL להגדרת מבנה (CREATE TABLE)

ג. DML להגדרת גישה, DDL לבקרת גישה

ד. אין הבדל מהותי ביניהם

התשובה הנכונה: ב

הסבר: DML (Data Manipulation Language) משמש לעבודה עם הנתונים עצמם, בעוד DDL (Data Definition Language) משמש להגדרת המבנה של בסיס הנתונים.

מקור במצגת: עמוד 7

---

#### 4. מהו היתרון העיקרי של SQLite?

---

א. תמיכה במשתמשים מרובים

ב. מהירות גבוהה במיוחד

ג. עצמאי ולא דורש שרת נפרד

ד. תמיכה בטרנזקציות מורכבות

התשובה הנכונה: ג

הסבר: SQLite הוא מנוע SQL עצמאי שלא דורש שרת נפרד, מה שהופך אותו לפתרון אידיאלי לאפליקציות מקומיות.

מקור במצגת: עמוד 18

---

#### 5. איזו פעולה מומלצת למניעת SQL Injection?

---

א. הצפנת כל הנתונים בבסיס הנתונים

ב. שימוש בשאילתות פרמטריות

ג. מחיקת כל המשתמשים הלא פעילים

ד. הגבלת אורך השאילתות

התשובה הנכונה: ב

הסבר: שימוש בשאילתות פרמטריות הוא הדרך המומלצת למניעת SQL Injection כי הפרמטרים מטופלים בנפרד מהשאילתה עצמה.

מקור במצגת: עמוד 24

סיכומים בנושאים שונים

סיכום פונקציות עם סיכון גבוהה והתמודדות איתן

פונקציה	בעיה אבטחתית	חולשה/סיכון	פתרון/המלצה
<code>strcpy</code>	לא בודקת גבולות הזיכרון.	Buffer Overflow - הצפת זיכרון. יכולת להזריק קוד זדוני.	השתמש ב- <code>strncpy</code> או ב- <code>strncpy</code> כדי להבטיח שהמיתר לא חורג מגבולות הזיכרון.
<code>sprintf</code>	לא בודקת גבולות הזיכרון.	Buffer Overflow - כתיבה מעבר למרחב הזיכרון המוקצה.	השתמש ב- <code>snprintf</code> שכוללת פרמטר המגביל את הגודל.
<code>gets</code>	קוראת למיתר מבלי לדעת את הגודל.	Buffer Overflow - הצפת זיכרון, חשש להזרקה או קריסת התוכנית.	השתמש ב- <code>fgets</code> במקום <code>gets</code> .
<code>scanf</code>	לא תמיד מבצעת את הבדיקה הנכונה על אורך הקלט.	Input Validation Issues - הצפת זיכרון או קריאה לא תקינה.	השתמש ב- <code>scanf</code> עם הגבלת אורך הקלט, לדוגמה <code>%9s</code> במקום <code>%s</code> .
<code>strcat</code>	לא בודקת גבולות הזיכרון.	Buffer Overflow - הצפת זיכרון, חשש להזרקת קוד זדוני.	השתמש ב- <code>strncat</code> עם הגבלת אורך המיתר.
<code>memcpy</code>	לא בודקת גבולות הזיכרון.	Buffer Overflow - חשש להצפה של זיכרון.	השתמש ב- <code>memmove</code> אם הזיכרון חופף, ודאג לבצע בדיקות גבולות ידניות.
<code>alloca</code>	לא בודקת גודל הזיכרון המוקצה, עשויה לגרום להקצאת יותר מדי זיכרון על הסטאק.	Stack Overflow - שימוש בזיכרון לא מבוקר, פגיעות לאובדן נתונים.	השתמש ב- <code>malloc</code> או <code>new</code> להקצאת זיכרון דינמי.
<code>free</code>	אם משתנים לא הוקצו בצורה נכונה, או לא שוחררו כראוי, יש סיכון לדליפות זיכרון.	Memory Leaks - לא שוחרר זיכרון, פגיעה בממשק מערכת.	השתמש ב- <code>delete</code> / <code>delete []</code> וודא שהזיכרון שוחרר כראוי.
<code>delete</code>	אם מנסה למחוק זיכרון שלא הוקצה או כבר שוחרר, עלולה לגרום לקריסת התוכנית.	Double Free - קריסה או פגיעות אחרות.	ודא שלא משחררים פעמיים את אותו זיכרון, השתמש ב- <code>nullptr</code> לאחר שחרור.
<code>system</code>	מאפשר הרצת פקודות מערכת חיצוניות.	Command Injection - אפשרות להריץ פקודות זדוניות.	הימנע משימוש ב- <code>system</code> אם אפשר, או השתמש ב-API בטוח יותר כמו <code>exec</code> .

- **Buffer Overflow:** כאשר מיתר או מבנה נתונים אחר חורגים מגבולות הזיכרון שהוקצה להם. זה יכול להוביל לשחיתות נתונים, קריסת תוכנה או התקפות זדוניות כמו הזרקת קוד.
- **Stack Overflow:** התרחשויות זיכרון לא נכונות בסטאק שמובילות לקריסת תוכנה או ניצול זיכרון בצורה מזיקה.
- **Memory Leaks:** זיכרון שנשאר מוקצה אך לא מנוהל כראוי, יכול להוביל למחסור בזיכרון ואובדן ביצועים לאורך זמן.
- **Double Free:** ניסיון לשחרר זיכרון פעמיים, דבר שיכול להוביל לקריסת התוכנית או לניצול פגיעות.
- **Command Injection:** מתקפה שמבצעת הזרקה של פקודות זדוניות לתוך מערכת ההפעלה דרך קריאה לפונקציות כמו `system`.

## סיכום תהליכי מנגנוני הצפנה אי סימטרי

### סיכום מנגנון ההצפנה והמפתח הציבורי

#### א. הצפנה באמצעות מפתח ציבורי (א-סימטרי)

- כל משתמש יוצר זוג מפתחות:
- **מפתח ציבורי (Public Key)** – משמש להצפנה.
- **מפתח פרטי (Private Key)** – נשמר בסוד ומשמש לפענוח.
- המפתח הציבורי מועבר לשולח דרך ערוץ לא-מאובטח.
- השולח מצפין את המידע עם המפתח הציבורי ושולח אותו.
- רק המקבל יכול לפענח את המידע עם המפתח הפרטי שלו.

#### חסרונות: ●

1. חשופ ל"איש בתווך" (MITM), שבה תוקף מחליף את המפתח הציבורי.
2. הצפנה א-סימטרית דורשת כוח חישובי גבוה, ולכן פחות יעילה להודעות ארוכות.

## SSL/TSL

## ב. סכמת ההצפנה הנפוצה באינטרנט (TLS/SSL)

כדי לשפר יעילות, משלבים הצפנה א-סימטרית וסימטרית:

1. צד ב' (שרת) יוצר זוג מפתחות (ציבורי ופרטי).
2. צד ב' שולח את המפתח הציבורי לצד א' (לקוח).
3. צד א' יוצר מפתח סימטרי (לשימוש בהצפנה מהירה).
4. צד א' מצפין את המפתח הסימטרי עם המפתח הציבורי של צד ב'.
5. צד ב' מפענח את ההודעה עם המפתח הפרטי שלו – כעת שני הצדדים מחזיקים באותו מפתח סימטרי.
6. מעכשיו התקשורת מוצפנת בהצפנה סימטרית, שהיא מהירה ויעילה יותר.

• למה זה עובד טוב?

- השלב הא-סימטרי (צעד 1-5) מגן על המפתח הסימטרי.
- לאחר מכן, הצפנה סימטרית מהירה משמשת לשאר התקשורת.

## עמודים חשובים בחוברת :

1. עמ 111 כתיבת שרת ולקוח CPP עם BOOST
2. עמ 117-119 שרת בפיתוח עם ובלי ריבוי לקוחות
3. עמ 150-151 כתיבה SQL עם פיתוח

## אופרטור העברה והשמה

מאפיין	אופרטור השמה רגיל	אופרטור השמה בהעברה
חתימה	<code>MyClass&amp; operator=(const MyClass&amp; other)</code>	<code>MyClass&amp; operator=(MyClass&amp;&amp; other) noexcept</code>
מטרה	העתקת תוכן מאובייקט אחד לאחר	העברת משאבים מאובייקט אחד לאחר
טיפול במשאבים	יוצר עותק חדש של המשאבים	"גונב" את המשאבים מהאובייקט המקורי
הקצאת זיכרון	מקצה זיכרון חדש	לא מקצה זיכרון חדש
יעילות	פחות יעיל - דורש העתקה מלאה	יעיל מאוד - רק מעביר מצביעים
אובייקט מקור	נשאר ללא שינוי	נשאר במצב תקין אך ריק (null)
שימוש נפוץ	העתקה של אובייקטים קבועים	העברת אובייקטים זמניים
זריקת חריגות	יכול לזרוק חריגות	בד"כ מסומן כ- <code>noexcept</code>
אופן הפעלה	<code>a = b</code>	<code>a = std::move(b)</code>
מצב אחרי פעולה	שני אובייקטים תקינים עם תוכן זהה	אובייקט יעד תקין, מקור ריק
צורך בבדיקת <code>self-assignment</code>	חייב לבדוק	מומלץ לבדוק
משאבים נדרשים	גבוהים (זמן + זיכרון)	נמוכים מאוד
תאימות לאחור	קיים מאז C++ המקורי	נסף ב-C++11
שימוש במימלים	<code>vector.push_back(obj)</code>	<code>or.emplace_back(std::move(obj))</code>

הבדל בין פונקציה רגילה לvirtual

פונקציה וירטואלית	פונקציה רגילה	מאפיין
<code>virtual void func()</code>	<code>void func()</code>	הגדרה
זמן ריצה	זמן קומפילציה	עיתי קבלת החלטות
דרך טבלה וירטואלית (vtable)	ישיר	מנגנון קריאה
מצביע vptr לכל אובייקט	אין	תקורת זיכרון
קטנה (חיפוש בטבלה)	אין	תקורת ביצועים
תקרא לפונקציה של המחלקה בפועל	תקרא לפונקציית הבסיס	שימוש במצביע לבסיס
כן, עם התנהגות פולימורפית	כן, אבל לא בצורה פולימורפית	אפשרות דריסה
<code>override</code> (מומלץ)	לא נדרש	סימון דריסה
פונקציות שצריכות להיות מותאמות במחלקות יורשות	פונקציות שלא צריכות להשתנות	מתאים ל...
כן (עם <code>0</code> )	לא	יכולה להיות טהורה
מחלקה עם פונקציה וירטואלית טהורה הופכת למופשטת	אין מיוחדת	השפעה על מחלקה
<code>calculateArea()</code>	<code>getID()</code>	דוגמה לשימוש
חובה אם טהורה	אופציונלי	מימוש במחלקה נגזרת
מעט יותר גבוהה	נמוכה	עלות קריאה
כן	לא	תמיכה בפולימורפיזם